

AnimusCore — Engineering Defense FAQ

Purpose: a peer-to-peer prep sheet for live conversations with quant devs and low-latency infra leads — technical review calls, issue-tracker threads, pre-pilot due diligence. Every claim below cites the exact source line it comes from. It complements, not duplicates, `docs/technical_eval/ARCHITECTURE.md` — that document has the full narrative and a comparison table against LMAX Disruptor / Boost.Interprocess / Aeron (§5); this one is built to be recalled mid-conversation, not read end to end.

Source files referenced (repo-relative paths): - `AnimusCore_v1/animus.hpp` — header-only core engine - `include/animus/shm_ipc.hpp` — cross-process `ShmRing<T>` / `SpmcRing<T>` - `include/animus/thread_affinity.hpp` — pinning, priority, cache-line constants - `tests/eval_harness.cpp` — external evaluation harness - `dist/AnimusCore_Eval_Sandbox/README_BENCHMARK.txt` — sandbox build/run/baseline - `bindings/animus_py.cpp` — nanobind Python binding - `docs/technical_eval/ARCHITECTURE.md`, `FINTECH_CHANGELOG.md`, `COMPATIBILITY_TUNING_GUIDE.md` — deeper narrative + measured benchmark history

Part 1 — The Core 5 Mechanical Primitives

1. Cache-Line Alignment & False Sharing

False sharing happens when two atomics written by *different* cores land on the same cache line: MESI/MOESI has to ping-pong that line's ownership (Modified → Invalid on the other core) on every write, even though the two fields are logically unrelated. The fix is structural — put anything a different core writes on its own line — not a runtime one.

- `TelemetryPayload` (`animus.hpp:68-75`) is `alignas(64)`, and two `static_assert`s pin both halves of the guarantee at compile time: `sizeof(TelemetryPayload) == 64` and `alignof(TelemetryPayload) == 64`. The four real fields (`timestamp_cycles`, `event_id`, `trace_id`, `metric_value`) occupy exactly the first 24 bytes with zero *internal* padding — `alignas(64)` is what pads `sizeof` up to a full line, not scattered gaps between fields.
- Contrast: `ThreatSignal` (`animus.hpp:84-91`) is deliberately **not** cache-line padded. It's copied byte-for-byte across the C-ABI into a caller's own struct (e.g. a ctypes `Structure`); padding it here without the caller mirroring that padding would silently corrupt the caller's buffer. Know both cases — a reviewer who only sees `alignas(64)` on one struct and asks "why not this one too" is testing whether you understand *why*, not just pattern-matching the annotation.
- The actual false-sharing-prevention pattern lives on the ring cursors, not the payload structs: `LockFreeRingBuffer::enqueue_pos_ / dequeue_pos_` (`animus.hpp:449-450`) and `SpscRingBuffer::head_ /`

`tail_` (`animus.hpp:529-530`) are each `alignas(kCachelineBytes)` — producer and consumer cursors on separate lines.

- `kCachelineBytes` (`animus.hpp:334-362`) prefers `std::hardware_destructive_interference_size` when the toolchain defines `__cpp_lib_hardware_interference_size`, falling back to a hardcoded 64 otherwise (some libstdc++ releases withhold the macro over an ABI-stability concern — a value that differs across TUs built with different `-march` flags would be a silent ODR violation).
- Cross-process version: `RingHeader` (`include/animus/shm_ipc.hpp:305-328`) splits into a read-only wire descriptor, a producer-owned line (`head` + producer pid/heartbeat), and a consumer-owned line (`tail` + consumer pid/heartbeat), each `alignas(ANIMUS_CACHE_LINE_SIZE)`. `static_assert(sizeof(RingHeader) % ANIMUS_CACHE_LINE_SIZE == 0, ...)` (`shm_ipc.hpp:335-337`) fails the build if a field addition ever breaks that. This matters *more* across a process boundary than in-thread: the two sides are often on different sockets, so false sharing there costs cross-socket coherency traffic, not just cross-core.
- Cache-line size itself isn't a universal constant: `thread_affinity.hpp:57-71` sets `ANIMUS_CACHE_LINE_SIZE` to 64 by default, 128 on `__aarch64__` / `_M_ARM64` (Apple Silicon and Neoverse server parts use 128-byte lines). Get this question and the honest answer is "compile-time default by target arch, not runtime-probed — override the macro yourself for a specific ARM part if you need exact hardware detection."

2. Power-of-Two Ring Buffer & Bitwise Masking

Every in-process and SHM ring is sized via `round_up_pow2()` (`animus.hpp:439-443`, `519-523`) so slot lookup is `index & mask_` (`mask_ = capacity_ - 1`) instead of `index % capacity`. On x86-64, `AND` against a register is a single-cycle ALU op; a 64-bit unsigned `idiv` is a genuinely slow instruction — commonly cited at 20-40+ cycles depending on microarchitecture and operand size, and it's *not* constant-time (dividend magnitude affects latency), which is itself disqualifying for a hot-path primitive where you want a fixed cycle cost per call, not a data-dependent one.

Where this appears: `LockFreeRingBuffer` (`pos & mask_`, `animus.hpp:387,411`), `SpscRingBuffer` (`head & mask_ / tail & mask_`, `animus.hpp:499,511`), `ShmRing<T>` (`head & header_->mask`, `shm_ipc.hpp:501` and the pop-side equivalent).

The nuance worth having ready: `animus::SharedTelemetryChannel` (`animus.hpp:769-778`) deliberately uses plain `%`, not `&`, even though every other ring in the file uses the mask trick. Reason: it's wire-compatible with a pure-Python implementation (`animus.shm.SharedTelemetryRing`) whose `create()` can be called with *any* capacity, not necessarily a power of two — using a bitmask there would silently index the wrong slot the moment the Python side picked a non-power-of-two size. That's the answer to "did you just cargo-cult the bitmask everywhere" — no, one primitive explicitly trades the cycle-count optimization for wire-format correctness, and that trade is documented at the call site, not accidental.

3. Memory Ordering & Acquire-Release Semantics

The pattern is identical across every ring in the codebase, in-process and cross-process: a **relaxed** load of your own cursor, an **acquire** load of the other side's cursor, and a **release** store to publish your own advance.

- `LockFreeRingBuffer::push / pop` (`animus.hpp:383-429`): the CAS on `enqueue_pos_ / dequeue_pos_` is `memory_order_relaxed` (it's only arbitrating between racing producers/consumers over *which* index they get, not publishing data), the per-cell `sequence` load is `memory_order_acquire`, and the post-write `sequence.store` is `memory_order_release`.
- `SpSCRingBuffer::push / pop` (`animus.hpp:493-514`) and `ShmRing<T>::try_push / try_pop` (`shm_ipc.hpp:497-504`, `553-560`): relaxed load of your own index, acquire load of the peer's index, release store of your own advance — no CAS at all, because there is exactly one writer per index.
- `push_overwrite`'s forced reclaim (`shm_ipc.hpp:537`) upgrades to `memory_order_acq_rel` on `tail.fetch_add` specifically because a concurrent `try_pop` might be advancing the same `tail` at the same moment — `acq_rel` keeps that race well-defined (tail ends up at least one slot further along either way) without needing a full `seq_cst` fence.

Why not just use `seq_cst` everywhere and stop worrying about it: `seq_cst` on x86 requires a full memory fence (`MFENCE`) or a locked RMW instruction on every store to establish a single total order across *all* `seq_cst` operations in the program — cost that buys you nothing here, because a producer/consumer ring only needs a pairwise acquire/release handshake on *this* cursor, not a global ordering against every other atomic in the process. Acquire/release is the release-consistency model: everything the producer wrote before its `release` store is guaranteed visible to a consumer's `acquire` load of that same variable — exactly the guarantee a ring buffer needs, at the cost of a compiler barrier plus (on x86) nothing extra at the hardware level, since x86's TSO already gives you acquire/release for free and only `seq_cst` needs the fence.

Have the `read_cycle_counter()` nuance ready too (`animus.hpp:303-323`): it's deliberately **not** `lfence` - serialized, on purpose, in the hot path (`record()` / `push()` / `record_batch()`). An unserialized `__rdtsc()` can retire slightly out of order — fine for stamping one timestamp with no bracket to keep accurate, and serializing it would add a real pipelining stall to every single call. Contrast with `tests/eval_harness.cpp:107-115`'s `rdtsc_serialized()`, which *does* wrap both reads in `_mm_lfence()` — because that call needs a precise bracketed interval (start, do work, end), a different requirement from a single per-event stamp. Same primitive, two call sites, two deliberately different serialization choices — that's the kind of distinction that signals you understand the tradeoff rather than having memorized "always `lfence` `rdtsc`."

4. Zero-Heap Allocations

Two separate guarantees, don't conflate them:

- **Structural:** ring backing storage (`cells_` in `LockFreeRingBuffer` / `SpSCRingBuffer`, the mapped SHM region in `ShmRing<T>`) is allocated once, at construction — `push()` / `pop()` touch only pre-existing memory. `static_assert(std::atomic<uint64_t>::is_always_lock_free, ...)` (`animus.hpp:749-751`, `shm_ipc.hpp:332-334`) additionally guarantees the cursor atomics themselves never fall back to a mutex/futex — which matters doubly for the SHM case, since a futex-backed atomic wouldn't even be valid across a process boundary.
- **Measured, not asserted by inspection:** `tests/eval_harness.cpp:56-83` overrides global `operator new / operator new[] / operator delete / operator delete[]` with atomic counters, snapshots them

immediately before the 1,000,000-iteration benchmark loop and reads them back immediately after (`eval_harness.cpp:218-244`) — any allocation during the timed region shows up as a nonzero delta, full stop, not "we believe this doesn't allocate." `README_BENCHMARK.txt:16` reports the actual result: **PASS, zero heap allocations** across that run. Ring creation and the harness's own `reserve()` calls happen strictly before that snapshot, so they can't contaminate the measurement.

If asked "what about exceptions / `std::bad_alloc` inside the hot path" — `push()` / `pop()` / `record()` / `record_batch()` are all `noexcept` and touch no allocating call; the only allocating paths in the engine (`add_rule` , `add_cep_rule`) are explicitly cold, mutex-guarded, copy-on-write snapshot swaps (`animus.hpp:1229-1276`), wrapped in `try` / `catch` so an allocation failure there leaves the existing rule set valid rather than corrupting state.

5. Invariant TSC Timing

`read_cycle_counter()` (`animus.hpp:324-332`) calls `__rdtsc()` directly (MSVC intrinsic or `__builtin_ia32_rdtsc()` on GCC/Clang), falling back to `std::chrono::steady_clock` only on non-x86 targets. Why raw RDTSC beats `std::chrono` / `clock_gettime` on the hot path: even where the kernel serves a `CLOCK_MONOTONIC` read from a VDSO page (no syscall trap), you're still going through a function call, a VDSO indirection, and library-level bookkeeping around clock source selection — a raw `RDTSC` intrinsic is a handful of cycles with no indirection at all. On Windows, `QueryPerformanceCounter` is frequently backed by the TSC anyway on modern hardware; raw `RDTSC` just skips that layer.

"Invariant TSC" is the property that makes raw cycle counts meaningful across a whole run: the counter ticks at a fixed rate regardless of core P-state (frequency scaling) or C-state (sleep-state) transitions — without it, a core that turbo-boosted mid-benchmark would silently skew every cycle count taken after the ramp. `tests/eval_harness.cpp:117-131` 's `has_invariant_tsc()` checks CPUID leaf `0x80000007` , EDX bit 8, and the harness prints whether it's present rather than assuming it (`eval_harness.cpp:169-170`). Practically: recalibrate raw cycles → ns per host (`README_BENCHMARK.txt:18` — "Approx ns... recalibrate TSC per host"), and set the CPU frequency governor to `performance` on any host you're benchmarking on, or C-state/P-state transitions reintroduce exactly the noise invariant TSC is supposed to remove (`docs/technical_eval/COMPATIBILITY_TUNING_GUIDE.md:34`).

Part 2 — The 10 Toughest Quant Lead Inquiries

1. "How do you handle multi-producer contention and cacheline bouncing?"

`LockFreeRingBuffer` (`animus.hpp:364-451`) is a Vyukov MPMC ring: each cell carries its own `sequence` atomic rather than a single shared "full" flag, so once N producers CAS past each other on the shared `enqueue_pos_` cursor, they immediately diverge into different cells and stop contending with each other. The CAS itself is the one point of real contention — that's inherent to any single-cursor MPMC design (moodycamel's producer-token model exists for the same reason). If you need to eliminate cross-producer bouncing entirely, don't fight the MPMC ring — use one `SpSCRingBuffer` per producer thread (`animus.hpp:480-531`) and fan in on the

consumer side. That's the actual ladder here: MPMC ring for simplicity when producer count is small or variable, N×SPSC when you want zero producer-side contention and can afford one dedicated consumer thread per fan-in point.

2. "What happens on ring overflow when a consumer lags behind?"

In-process rings are bounded-only: `push()` returns `false` on a full ring (`animus.hpp:396` , `420` , `497-498`) and the caller decides what to do — retry, drop, spill elsewhere. The SHM layer (`ShmRing<T>`) gives you an explicit choice instead of one hardcoded policy: `try_push()` (`shm_ipc.hpp:493-504`) is the same bounded-refusal contract, while `push_overwrite()` (`shm_ipc.hpp:506-539`) never blocks and never refuses — on a full ring it force-reclaims the oldest unconsumed slot via `tail.fetch_add(1, acq_rel)` and increments a counted `dropped_count()` . That reclaim is a documented, accepted race if a consumer is mid- `pop()` on the exact slot being reclaimed (a torn read at that one boundary) — so `push_overwrite` is scoped to consumers that tolerate that (telemetry, sampling, latest-value snapshots), never to a channel that must see clean records. `SpmcRing<T>` (broadcast, covered in `ARCHITECTURE.md` §5) takes a third approach — no backpressure concept at all; each consumer independently detects its own overrun and self-corrects.

3. "Why `-O3 -march=native -DNDEBUG` , and doesn't `-DNDEBUG` strip your safety checks?"

Exact build line is in `README_BENCHMARK.txt:10-13` . `-DNDEBUG` only strips the standard `assert()` macro — and the hot path (`push` / `pop` / `record` / `record_batch`) contains zero runtime `assert()` calls to strip. Every layout invariant in this codebase is a `static_assert` (`sizeof` / `alignof` checks on `TelemetryPayload` , `SharedTelemetryRecord` , `SharedRingHeader` , `RingHeader` , `L2Update` , `TradeTick` , `LicensePayload` — e.g. `animus.hpp:74-75` , `730-731` , `747-751` , `1462` , `1482` , `909` ; `shm_ipc.hpp:332-337`), which is a compile-time check with zero runtime cost, unaffected by `NDEBUG` either way. So the honest framing is: `-DNDEBUG` is a standard release flag here, not "we turn off safety checks in production" — there weren't runtime checks on this path to turn off.

4. "What's the real Python/C++ interop latency — zero-copy via nanobind?"

Two separate paths exist, pick based on what you actually need:

- **C-ABI / ctypes** (`animus.hpp` 's `extern "C"` block, `animus/bindings.py`) — stable ABI, works from any language with a C FFI, cross-process safe. Pays ctypes call-marshalling cost per call; the `*_batch` functions (`animus_record_events_batch` , `animus_spsc_record_events_batch`) exist specifically to amortize that cost over N events per call instead of paying it per event.
- **nanobind** (`bindings/animus_py.cpp:1-28`) — in-process only, `#include` s `animus.hpp` directly and drives `animus::SpscRingBuffer<TelemetryPayload>` with no C-ABI boundary at all. `drain()` pops events into a scratch buffer the binding owns (one memcpy-equivalent per event — unavoidable at any producer/consumer handoff), then exposes that same scratch memory to Python via the buffer protocol: zero-copy specifically means no per-event Python object is constructed and no *second* copy is made to hand the batch to Python — the returned view aliases the scratch buffer and is only valid until the next `drain()` call. Chosen over pybind11 specifically for lower per-call dispatch overhead (`animus_py.cpp:12-16`) — that's a build-time comparison to verify on your own workload, not a number we're asserting here.

If you're already in-process and latency-sensitive, skip both bindings and drive `animus::Engine / SpscRingBuffer` directly from C++ — the binding layers exist for callers that need Python or process isolation, not because every path routes through them.

5. "CPU core isolation, thread pinning, NUMA topology?"

Pinning: `animus_pin_current_thread_to_core` (`SetThreadAffinityMask` / `pthread_setaffinity_np`, `animus.hpp:1631-1638`) and `animus_pin_current_thread_to_core_exclusive` (`thread_affinity.hpp`), which pins *and* raises the thread's scheduling priority (`THREAD_PRIORITY_TIME_CRITICAL` on Windows; `SCHED_FIFO` with a `nice(-20)` fallback on Linux). Use the exclusive variant, always — `FINTECH_CHANGELOG.md` Phase 14 documents a real regression: plain pinning without priority elevation made p99.99 tail latency *worse* than unpinned in most trials (up to 5-6x), because a pinned-but-not-prioritized thread has nowhere to migrate under contention. The exclusive variant reversed that in 15/15 re-run trials and is what `COMPATIBILITY_TUNING_GUIDE.md`'s published pinned numbers (P50 SPSC ingest 55.39ns → 19.43ns, P99 tick-to-telemetry 398,189ns → 56.22ns) actually use.

NUMA: **not automated, and I won't claim otherwise.** There's no NUMA-aware allocator or topology query in this codebase — allocating the ring's backing memory on the correct node and pinning producer/consumer to that same node is an operator responsibility (`numactl` on Linux, Windows NUMA APIs), per `COMPATIBILITY_TUNING_GUIDE.md:33`. A ring spanning NUMA nodes turns every cache-line transfer into a cross-socket hop — that's on you to avoid at deployment time, not something AnimusCore detects or corrects for you.

6. "Drop policies and backpressure under burst traffic?"

Three tiers, pick per channel: bounded refusal (`try_push` / `push_spin` — full ring returns `false`, producer decides), decoupled lossy (`push_overwrite` — never blocks, drop-oldest with a counted `dropped_count()`), and SPMC broadcast (no backpressure concept at all — `broadcast()` always publishes, each consumer detects its own overrun and jumps forward, tallying its own `overrun_count()`; see `ARCHITECTURE.md` §5 for the full mechanics). Separately, the persistence worker (`EngineImpl::process_persistence_queue`, `animus.hpp:1082-1168`) surfaces backpressure to the *producer* rather than silently discarding on a sink failure: if the disk log can't be opened, events stay in the ring and `record()` starts returning `false` once it fills, instead of the worker quietly dropping events while the sink is down.

7. "Kernel bypass networking — Solarflare EF_VI, DPDK?"

No built-in NIC driver integration, and that's a scope boundary, not a gap I'm hiding. `MarketDataFeed` (`animus.hpp:1404-1573`, C-ABI at 1730+) is producer-agnostic: `push_12_update()` / `push_trade()` just need *something* to call them with already-decoded fields, and they're safe for any number of concurrent producer threads (same Vyukov MPMC ring underneath). A DPDK RX thread or an EF_VI-pollled thread that owns your packet decode calls `push_12_update()` as the last step after parsing — same pattern as any other producer, no adapter code needed on our side, no extra serialization step in between decode and ingest. We sit downstream of whatever kernel-bypass fabric you're already running; we don't ship the NIC driver layer.

8. "Behavior across heterogeneous cores — Intel P-cores vs E-cores?"

No automatic hybrid-topology detection — `animus_get_cpu_count()` (`animus.hpp:1667-1669`) reports total logical CPU count only, not core type, and there's no CPUID-based P/E classification wired into the pinning API. This actually bit the project directly: `FINTECH_CHANGELOG.md` Phase 14 documents pinning to "the highest-numbered core" (a common informal convention) on a hybrid Intel dev machine, which turned out to be an Efficiency core, and measured **34.5x worse** tail latency than not pinning at all. Current guidance, and what the shipped benchmark tooling actually does: probe candidate cores empirically with a cheap workload and pin to whichever measures fastest (`benchmarks/fintech_tail_latency.py`'s approach), or query OS/vendor topology yourself (Windows `GetLogicalProcessorInformationEx`, Linux `/sys/devices/system/cpu/cpuX/topology`, CPUID leaf `0x1A` on Intel hybrid parts) before calling `pin_current_thread_to_core_exclusive`. The published pinning benchmark in `COMPATIBILITY_TUNING_GUIDE.md` was run on exactly this kind of hybrid part (i7-14650HX, 24 logical cores) — the numbers we quote already reflect this reality rather than a clean, homogeneous-core lab result.

9. "Crash resilience and shared-memory persistence?"

Segment survives a process crash, not a reboot. Windows: named mappings are pagefile-backed and `HANDLE`-refcounted — the OS closes a crashed process's handles for it, so the mapping is destroyed only once every process's handle (including the dead one's) is gone (`animus.hpp:580-623`). POSIX: the `/dev/shm` node persists until explicitly `shm_unlink`ed — a killed producer leaves it in place for a surviving consumer to keep reading.

Dead-peer detection is pid-liveness-based, not signal-based — `SIGKILL` runs no handler at all, so there's nothing to trap. `is_process_alive()` (`kill(pid,0)` on POSIX; `OpenProcess` + exit-code check on Windows, `shm_ipc.hpp:602-664`) is the actual mechanism; `mark_producer_attached()` / `producer_heartbeat()` layer a staleness check on top for the complementary case — a peer that's alive but wedged (stopped under a debugger, deadlocked), not dead.

Mid-write crashes don't produce torn records through the normal path: a producer killed between writing a slot and its `head.store(..., release)` never advances `head`, so a consumer's `acquire`-load of `head` never observes that in-flight slot — that's the same release/acquire guarantee from Part 1 §3, working here as a crash-consistency property, not just a performance one. (This does not cover the already-separate, opt-in `push_overwrite` torn-read race — different mechanism, different accepted tradeoff.) Durable disk persistence is a separate layer: `EngineImpl::process_persistence_queue` flushes after every batch (`animus.hpp:1148-1153`), bounding data loss on a hard crash to whatever was in the one in-flight batch, not the ring's whole history.

10. "Why AnimusCore over `moodycamel::ReaderWriterQueue` or an LMAX Disruptor port?"

Not a raw-cycle-count superiority claim — I have not run a head-to-head benchmark against `moodycamel` in this repo, and I won't manufacture one on the spot. `moodycamel` and Disruptor are both excellent, proven single-process queues. The actual differentiator is scope: AnimusCore isn't "a faster queue," it's a queue plus genuine cross-process transport with wire-format and schema versioning checked at `open()` time (`shm_ipc.hpp:439-471`), pid + heartbeat crash detection, a CEP rule engine evaluating sliding-window aggregates directly on the ingest hot path (`CepRuleState`, `animus.hpp:190-301`), license-gated core pinning,

and a nanobind zero-copy Python binding — all built on the same struct/ABI conventions, as one coherent stack instead of something you'd assemble yourself out of moodycamel + a hand-rolled SHM wrapper + your own pinning code + your own Python bindings. Disruptor specifically has no cross-process story at all — its ring lives in one JVM's heap. `ARCHITECTURE.md` §5 has the full side-by-side table against Disruptor / Boost.Interprocess / Aeron if they want it. The honest pitch is integration and ownership, not a benchmark claim — don't get pulled into defending a number that was never measured.

Part 3 — The 5-Minute Founder Drill

- **Cache line size:** 64B on x86_64 and most ARM; 128B on `aarch64` / Apple Silicon / Neoverse (`thread_affinity.hpp:57-71`) — compile-time default by target, not runtime-probed.
- **Ring sizing:** always power-of-two, `mask = capacity - 1`, `index & mask` (1 cycle) not `idiv` (20-40+ cycles, non-constant-time). One deliberate exception: `SharedTelemetryChannel` uses `%` for Python wire-format compatibility with arbitrary capacities (`animus.hpp:769-778`).
- **Memory order:** relaxed-own / acquire-peer / release-publish on every handoff, in-process and cross-process alike. Zero `seq_cst`, zero locked instructions, zero `MFENCE` on the hot path. `read_cycle_counter()` is deliberately *not* lfence-serialized (single stamp, not a bracketed interval) — `eval_harness.cpp`'s own RDTSC reads *are* lfence-serialized, because that one measures an interval. Know which is which.
- **Zero heap:** measured, not asserted — `operator new` / `delete` override + before/after snapshot around a 1,000,000-iteration loop (`tests/eval_harness.cpp`), confirmed PASS in `README_BENCHMARK.txt`.
- **TSC:** raw `__rdtsc()`, invariant TSC = fixed rate across P-state/C-state transitions (CPUID leaf `0x80000007`, EDX bit 8) — recalibrate cycles→ns per host, set the `performance` governor before benchmarking.
- **Pinning:** always `pin_current_thread_to_core_exclusive`, never plain pinning alone — plain pinning without priority elevation measured *worse* p99.99 than no pinning at all (`FINTECH_CHANGELOG.md` Phase 14).
- **NUMA / hybrid cores:** not automated in either case — operator's job via `numactl` / topology APIs, and empirical core-probing for P-core/E-core selection. Say this plainly; don't imply auto-awareness that doesn't exist.
- **Kernel bypass (DPDK/EF_VI):** not our layer. `MarketDataFeed` takes already-decoded fields from whatever thread calls `push_l2_update()` / `push_trade()` — a DPDK or EF_VI RX thread is just another producer.
- **Drop policy:** `try_push` = bounded refusal; `push_overwrite` = lossy drop-oldest, counted, documented torn-read boundary; SPMC broadcast = no backpressure, per-consumer self-correcting overrun.
- **Build:** `-O3 -march=native -DNDEBUG`. Hot path has zero runtime `assert()` to strip — every layout invariant is a compile-time `static_assert`.

- **Never cite the eval sandbox for:** sub-100ns tick-to-trade or 16.5M+ pushes/sec multi-producer throughput — those are separate integration-suite figures (different code paths, different concurrency), explicitly called out as out-of-scope for this sandbox in `README_BENCHMARK.txt:22-28`.
- **moodycamel/Disruptor question:** answer with architecture + integration scope, never a benchmark number that hasn't actually been run.